

Documentación Técnica: Sistema de Adquisición y Procesamiento de Señales (SDR-DSP)

Grupo de Control y Procesamiento Digital de Señales

21 de diciembre de 2025

Resumen

Este documento detalla la implementación técnica de un sistema de adquisición basado en hardware SDR y procesamiento digital en C. Se describen los módulos de filtrado en el dominio de la frecuencia mediante FFTW3, los mecanismos de demodulación FM con deénfasis y el sistema de configuración dinámica. Se incluye la actualización del motor PFB v2 con corrección IQ y el marco de pruebas automatizadas mediante `pytest`.

Índice

1. Arquitectura del Nodo de Sensor y Estrategias de Adquisición	4
1.1. Orquestación y Máquina de Estados	4
1.2. Gestión de Memoria y Persistencia Segura	5
1.3. Estrategias Avanzadas de Adquisición Espectral	5
1.3.1. Estrategia 1: Offset & Crop (Modo Realtime)	5
1.3.2. Estrategia 2: Spectral Stitching (Modo Campaña)	6
1.4. Fiabilidad y Tolerancia a Fallos	6
1.5. Configuración del Backend RF	7
2. Implementación del Motor de Adquisición y Procesamiento IQ	7
2.1. Capa de Abstracción de Hardware (HAL) y Sintonía Robusta	7
2.2. Gestión de Memoria mediante Buffers Circulares	8
2.3. Procedimientos de Procesamiento Digital de Señales (DSP)	8
2.3.1. Normalización y Conversión de Formato	8
2.3.2. Dimensionamiento Dinámico de la FFT	8
2.3.3. Cálculo de Densidad Espectral de Potencia (PSD)	9
2.3.4. Escalado a Unidades Físicas (dBm)	9
3. Procesamiento Digital de Señales y Arquitectura de Tiempo Real	9
3.1. Justificación del Diseño en Sistemas Embebidos	9
3.2. Cadena de Demodulación y Métricas (AM/FM)	10
3.2.1. Preprocesamiento IQ	10
3.2.2. Algoritmos de Demodulación	10
3.2.3. Postprocesamiento y Métricas de Telemetría	11
3.3. Métodos de Estimación Espectral de Alta Resolución	11
3.3.1. Método de Welch (Periodograma Promediado)	11

3.3.2.	Banco de Filtros Polifásico (PFB - Polyphase Filter Bank)	12
3.3.3.	Conversión a Unidades Logarítmicas (dBm)	12
3.4.	Corrección de Respuesta (Blind Equalization)	13
4.	Postprocesamiento Espectral para Cumplimiento Normativo	13
4.1.	Idea clave: una PSD no “dice” sola si hay infracción	13
4.2.	Diagrama de flujo del módulo de cumplimiento	14
4.3.	Definiciones y variables (para leer el resto sin perderse)	15
4.4.	Entrada: qué necesita el algoritmo y por qué	15
4.5.	Rejilla de frecuencias: por qué se calcula y cómo afecta todo	15
4.6.	Estimación del piso de ruido mediante FCME (qué hace y por qué es apropiado)	16
4.6.1.	Problema que resuelve	16
4.6.2.	Conversión a escala lineal: por qué se hace	16
4.6.3.	Parámetros FCME (M_0 y α) con explicación intuitiva	16
4.6.4.	FCME paso a paso (con lectura humana)	16
4.7.	Del piso de ruido al umbral de detección: controlar falsos positivos y negativos	17
4.7.1.	Por qué existe Δ_{dB}	17
4.8.	Detección de emisiones: segmentación (cómo se define una emisión)	17
4.8.1.	Segmentación por contigüidad	17
4.8.2.	Filtros de robustez (para evitar artefactos)	18
4.9.	Medición por emisión: f_c , BW y potencia (y por qué se contemplan)	18
4.9.1.	Frecuencia central f_c (cómo se define y qué significa)	18
4.9.2.	Ancho de banda BW_{-3dB} (qué mide realmente)	18
4.9.3.	Potencia integrada P (por qué no basta el pico)	19
4.10.	Cruce con licencias por DANE: por qué se hace así	19
4.10.1.	Por qué se filtra por DANE	19
4.10.2.	Cómo se asocia una emisión a una licencia (criterio principal: frecuencia)	19
4.11.	Reglas de cumplimiento normativo (qué se evalúa y cómo se interpreta)	20
4.11.1.	Por qué se contemplan tolerancias	20
4.11.2.	Banderas de salida: hacer el resultado explicable	20
4.12.	Salida estructurada (JSON): qué contiene y por qué	20
5.	Anexo Metrológico: Caracterización y Validación Estadística del Sistema ANE-HX	21
5.1.	Determinación de la Sensibilidad y Caracterización del DANL	21
5.2.	Exactitud de Frecuencia y Estabilidad del Oscilador Local	21
5.3.	Error de Potencia y Protocolo de Calibración Metrológica	22
5.4.	Linealidad y Rango Dinámico Operativo	22
6.	Interfaz de Control y Configuración	23
6.1.	Estructura de Comandos JSON	23
7.	Testing (Pruebas Automatizadas)	23
7.1.	Propósito	23
7.2.	Alcance y limitaciones	23
7.3.	Entorno de ejecución	24
7.4.	Comandos de ejecución	24

7.5.	Estrategia de pruebas	24
7.6.	Resultados	25
7.7.	Conclusiones	25
8.	Infraestructura de Gestión y Aprovisionamiento Remoto OTA-ANE v7	25
8.1.	Justificación Técnica del Diseño	25
8.2.	Implementación y Despliegue del Sistema	26
8.2.1.	Configuración del Servidor (HUB)	26
8.2.2.	Aprovisionamiento del Sensor (Bootstrap)	27
8.3.	Gestión Administrativa y Diagnóstico	27
8.4.	Mantenimiento y Errores Comunes	27
9.	Lógica de Plataforma	27
9.1.	Arquitectura Funcional y Flujo de Valor	27
9.2.	Lógica de Monitoreo y Salud del Hardware	28
9.3.	Automatización y Rigor en las Campañas	28
9.4.	Continuidad de Operación (Redes Críticas)	28
10.	Glosario de Términos	29

1. Arquitectura del Nodo de Sensor y Estrategias de Adquisición

El diseño del nodo de sensor implementa una arquitectura híbrida y desacoplada, dividida en dos planos funcionales: un **Plano de Control** (*Control Plane*) desarrollado en Python 3, encargado de la orquestación lógica, comunicación con la nube y gestión de estados; y un **Plano de Datos** (*Data Plane*) de alto rendimiento desarrollado en C99, responsable del procesamiento digital de señales (DSP) y la interacción directa con el hardware de radiofrecuencia (SDR).

Esta separación permite combinar la flexibilidad de Python para la lógica de negocio y conectividad HTTP, con la eficiencia de C para el cálculo intensivo de FFT y el manejo de buffers de memoria.

1.1. Orquestación y Máquina de Estados

El núcleo operativo del sistema reside en el módulo **Orchestrator**, el cual implementa una máquina de estados finitos que gobierna el comportamiento del dispositivo. Basado en la clase **GlobalSys** del firmware, el sistema transita entre los siguientes estados operativos:

1. **IDLE (Reposo):** Estado por defecto de bajo consumo. El sistema espera comandos activos o la apertura de una ventana de tiempo programada (*Cron*).
2. **REALTIME (Tiempo Real):** Modo de baja latencia para visualización en vivo. Prioriza la velocidad de transmisión sobre el almacenamiento local, aplicando técnicas de recorte espectral al vuelo.
3. **CAMPAIGN (Campaña):** Ejecución de tareas de medición espectral programadas. En este estado, la prioridad es la integridad de los datos y la persistencia en disco.
4. **CALIBRATING (Calibración):** Rutinas de estabilización del PLL (*Phase Locked Loop*) y ajuste de ganancias (LNA/VGA) previas a una adquisición crítica.
5. **ERROR:** Estado de fallo seguro (*fail-safe*) ante excepciones críticas de hardware o software no recuperables.

La comunicación entre el Plano de Control (Python) y el Motor RF (C99/C++) se realiza mediante sockets **ZeroMQ (ZMQ)** de tipo **PAIR** sobre el protocolo IPC (*Inter-Process Communication*), lo que garantiza una latencia mínima y aislamiento de fallos entre procesos.

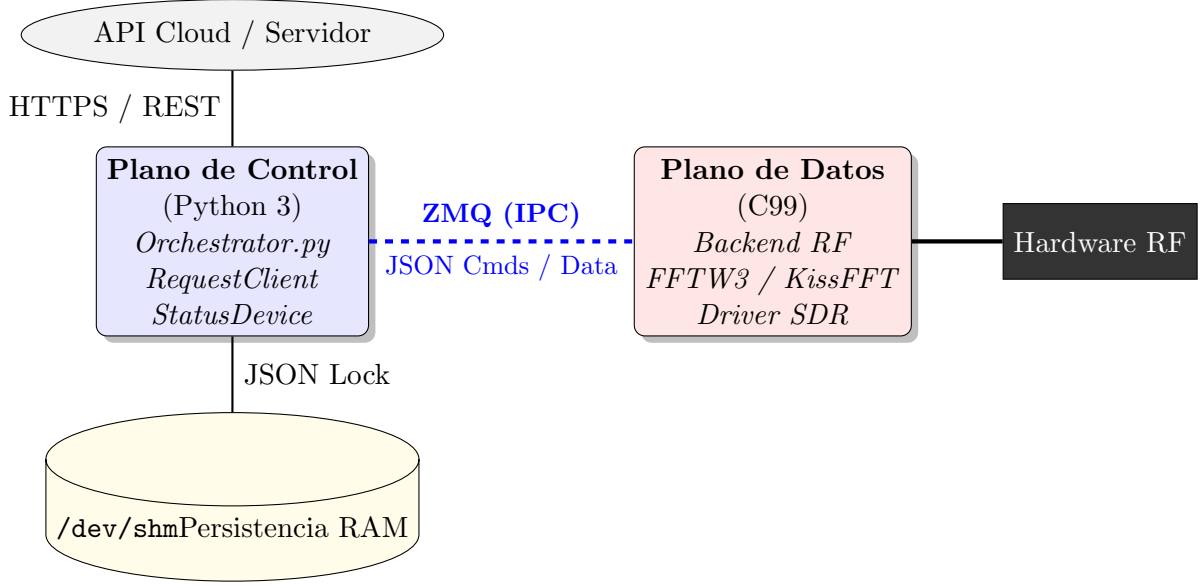


Figura 1: Arquitectura híbrida del sistema: Orquestación en Python y Procesamiento en C.

1.2. Gestión de Memoria y Persistencia Segura

Dado que el sistema opera en un entorno embebido (Raspberry Pi), se implementan estrategias para mitigar el desgaste de la tarjeta SD y asegurar la integridad de datos ante cortes de energía:

- **Almacenamiento Volátil (ShmStore):** Se utiliza `/dev/shm` (memoria compartida en RAM) para almacenar variables de estado de alta frecuencia. El acceso está protegido mediante bloqueo de archivos (*file locking* con `fcntl`), permitiendo concurrencia segura entre el orquestador y los scripts de ejecución.
- **Escritura Atómica en Disco:** El módulo de E/S implementa la función `atomic_write_bytes`. Esta rutina escribe primero los datos en un archivo temporal, fuerza el vaciado del buffer al medio físico (`os.fsync`) y finalmente realiza un reemplazo atómico del archivo destino. Esto evita la corrupción de archivos JSON si el sistema pierde energía durante la escritura.

1.3. Estrategias Avanzadas de Adquisición Espectral

Un desafío inherente a los receptores SDR de conversión directa es la presencia de un pico de corriente continua (*DC Spike*) en el centro de la banda base, generado por el automezclado del oscilador local. El sistema implementa dos algoritmos de software (definidos en `functions.py`) para eliminar este artefacto según el modo de operación:

1.3.1. Estrategia 1: Offset & Crop (Modo Realtime)

Utilizada cuando el ancho de banda solicitado (BW_{req}) es menor al ancho de banda máximo del hardware ($BW_{hw} \approx 20$ MHz). El procedimiento es el siguiente:

1. El oscilador local (LO) se sintoniza intencionalmente desplazado: $f_{lo} = f_c + \Delta f$, donde $\Delta f = 1$ MHz.

2. Como resultado, el pico DC aparece desplazado +1 MHz respecto al centro de interés.
3. Se aplica un algoritmo de limpieza (`SimpleDCSpikeCleaner`) que interpola ruido gaussiano basado en la varianza local (σ^2) de los *bins* vecinos para ocultar el pico.
4. Finalmente, se recorta digitalmente la banda para entregar al usuario exactamente la frecuencia central solicitada, libre de artefactos centrales.

1.3.2. Estrategia 2: Spectral Stitching (Modo Campaña)

Para mediciones de alta precisión donde la interpolación no es aceptable, la clase `AcquireCampaign` ejecuta una técnica de costura espectral:

1. **Captura A:** Se adquiere el espectro en la frecuencia central objetivo f_c .
2. **Captura B:** Se adquiere inmediatamente una segunda muestra desplazada $f_c + 2$ MHz.
3. **Patching (Parcheo):** El sistema identifica la zona central contaminada de la Captura A y la reemplaza quirúrgicamente con los datos limpios correspondientes de la Captura B, eliminando el pico DC sin perder información real de la señal.

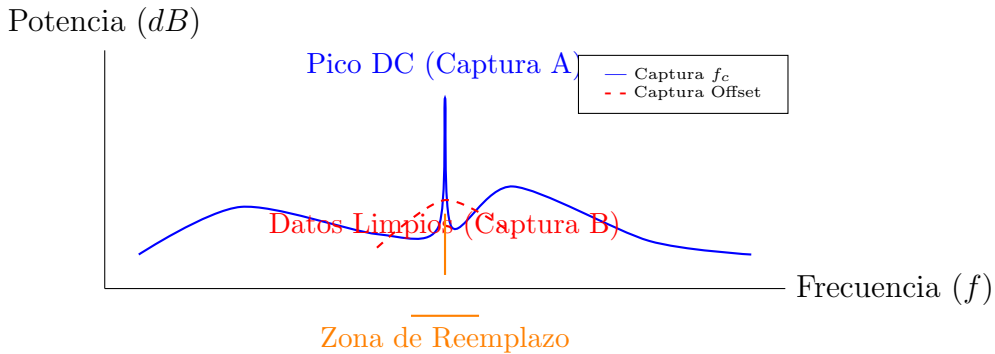


Figura 2: Técnica de *Spectral Stitching*: Reemplazo del pico DC mediante doble adquisición y superposición.

1.4. Fiabilidad y Tolerancia a Fallos

El sistema incorpora mecanismos de recuperación automática gestionados por el orquestador y los demonios del sistema (*daemons*):

- **Cola de Reintentos (*Retry Queue*):** Si la subida de datos a la nube falla por errores de red (códigos HTTP 5xx o *Timeouts*), los datos se serializan en el disco local dentro del directorio `/Queue`. El proceso `retry_queue.py` se ejecuta periódicamente intentando reenviar los archivos más antiguos primero (FIFO), eliminándolos solo tras una confirmación exitosa (HTTP 200).
- **Monitor de Estado (*Watchdog*):** El módulo `status_util.py` recopila métricas críticas como temperatura de CPU, uso de *swap*, latencia de red (*ping*) y deriva del reloj NTP. Estos datos se envían como *heartbeats*, permitiendo al servidor central diagnosticar la salud remota del sensor.

1.5. Configuración del Backend RF

La comunicación entre Python y C se realiza mediante estructuras JSON estrictas validadas por la clase `ServerRealtimeConfig`. Los parámetros principales configurables dinámicamente son:

Parámetro	Tipo de Dato	Descripción Técnica
<code>center_freq_hz</code>	<code>uint64</code>	Frecuencia central de sintonía (1 MHz – 6 GHz).
<code>sample_rate_hz</code>	<code>uint32</code>	Tasa de muestreo (define el ancho de banda instantáneo).
<code>method_psd</code>	<code>string</code>	Algoritmo espectral: <code>'welch'</code> (periodograma promediado) o <code>'pfb'</code> (banco de filtros polifásicos).
<code>window</code>	<code>string</code>	Ventana de apodización FFT (p. ej., Hann, Hamming, Blackman).
<code>overlap</code>	<code>float</code>	Porcentaje de solapamiento entre segmentos FFT (0,0 – 1,0).
<code>lna_gain</code>	<code>int</code>	Ganancia del Amplificador de Bajo Ruido (<i>frontend</i>).
<code>vga_gain</code>	<code>int</code>	Ganancia del Amplificador de Ganancia Variable (banda base).

Tabla 1: Parámetros de configuración enviados vía ZMQ al motor DSP.

2. Implementación del Motor de Adquisición y Procesamiento IQ

El motor de radiofrecuencia (*RF Engine*) desarrollado en C99 constituye la capa de ejecución de alto rendimiento del sistema. Su diseño está orientado a la gestión eficiente de recursos de hardware mediante el uso de hilos de ejecución concurrentes (*POSIX Threads*) y operaciones de memoria de baja latencia.

2.1. Capa de Abstracción de Hardware (HAL) y Sintonía Robusta

La interacción con el hardware SDR (HackRF One) se realiza a través de una capa de abstracción que encapsula la librería `libhackrf`. El sistema no solo inicializa el periférico, sino que implementa una rutina de **sintonía compensada por deriva térmica**.

- **Gestión de Ganancias:** Se controlan tres etapas independientes para optimizar el rango dinámico: el amplificador de bajo ruido (LNA) hasta 40 dB, el amplificador de ganancia variable (VGA) hasta 62 dB y un paso final de amplificación de RF (Amp) conmutable.

- **Corrección PPM:** Dado que los cristales osciladores presentan variaciones por temperatura, el sistema aplica una corrección de partes por millón (PPM) sobre la frecuencia objetivo f_{target} para obtener la frecuencia de sintonía real f_{tune} :

$$f_{tune} = f_{target} \cdot \left(1 + \frac{PPM_{err}}{10^6} \right) \quad (1)$$

- **Recuperación Automática:** El módulo `recover_hackrf` implementa un ciclo de re-inicialización en caso de pérdida de sincronía del bus USB, asegurando la continuidad de la operación en despliegues remotos.

2.2. Gestión de Memoria mediante Buffers Circulares

Para evitar la pérdida de muestras (*sample drops*) causada por la latencia del sistema operativo, se ha implementado un **Ring Buffer** concurrente de 100 MB.

- **Escritura No Bloqueante:** El *callback* de recepción de la librería HackRF vuelca los datos IQ crudos (*int8_t*) en la cabeza (*head*) del buffer circular de forma ininterrumpida.
- **Consumo Desacoplado:** Un hilo de procesamiento independiente extrae los datos desde la cola (*tail*). En el modo de audio activo, se utilizan dos punteros de lectura independientes para alimentar simultáneamente el hilo de telemetría espectral y el hilo de demodulación de audio, garantizando que el cálculo intensivo de la FFT no introduzca *jitter* en la salida de audio.

2.3. Procedimientos de Procesamiento Digital de Señales (DSP)

El motor realiza una secuencia de transformaciones matemáticas sobre las muestras IQ para convertirlas en información espectral inteligible:

2.3.1. Normalización y Conversión de Formato

Las muestras recibidas desde el hardware son pares entrelazados de 8 bits. El motor los normaliza al rango $[-1,0, 1,0]$ y los convierte al tipo complejo de doble precisión definido por la norma C99:

$$x[n] = \frac{I[n]}{128,0} + j \frac{Q[n]}{128,0} \quad (2)$$

2.3.2. Dimensionamiento Dinámico de la FFT

El sistema ajusta el tamaño de la ventana de procesamiento (N_{perseg}) de forma dinámica para cumplir con la resolución espectral (RBW) solicitada por el usuario. Se utiliza una lógica de redondeo a la potencia de dos superior para maximizar la eficiencia de la librería **FFTW3**:

$$N_{perseg} = 2^{\lceil \log_2 \left(\frac{ENBW \cdot F_s}{RBW_{req}} \right) \rceil} \quad (3)$$

Donde $ENBW$ representa el factor de ancho de banda equivalente de ruido de la ventana de apodización seleccionada (p. ej., 1.36 para una ventana Hamming).

2.3.3. Cálculo de Densidad Espectral de Potencia (PSD)

El motor permite conmutar entre dos métodos de estimación espectral:

- **Método de Welch:** Divide la señal en segmentos solapados, aplica una ventana (*Windowing*) para reducir el *spectral leakage* y promedia los periodogramas resultantes para reducir la varianza del ruido.
- **PFB (Polyphase Filter Bank):** Implementa una etapa de filtrado FIR polifásico antes de la FFT. Este método es superior en la detección de señales espurias y mejora el aislamiento entre canales adyacentes al aplicar un filtro prototipo Kaiser con un parámetro de atenuación $\beta = 8,6$.

2.3.4. Escalado a Unidades Físicas (dBm)

Tras el cálculo de la magnitud cuadrada de la transformada, los datos se someten a una operación de **FFT-Shift** para centrar la componente DC. Finalmente, se aplica una conversión a escala logarítmica asumiendo una impedancia de referencia de 50Ω :

$$P_{dBm}(k) = 10 \log_{10} \left(\frac{|X(k)|^2}{Z_0 \cdot 10^{-3}} \right) \quad (4)$$

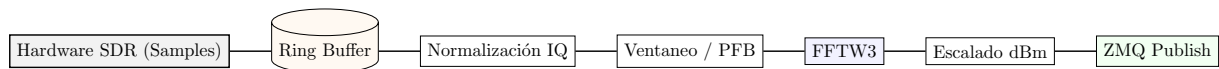


Figura 3: Diagrama de flujo del procesamiento de datos en el motor de radiofrecuencia.

3. Procesamiento Digital de Señales y Arquitectura de Tiempo Real

El subsistema de DSP constituye el núcleo crítico del firmware. Su diseño responde a un desafío de ingeniería fundamental: procesar un flujo de datos de alta velocidad (20 MS/s, equivalente a ≈ 40 MB/s de datos crudos IQ de 8 bits) en una plataforma de recursos limitados (Raspberry Pi), garantizando simultáneamente la visualización espectral de alta resolución y la transmisión de audio de baja latencia hacia un servidor remoto.

3.1. Justificación del Diseño en Sistemas Embebidos

A diferencia de los enfoques convencionales basados en *frameworks* pesados (e.g., GNU Radio), este sistema implementa un motor nativo en C99 altamente optimizado. Las decisiones de diseño se justifican por tres restricciones clave:

- **Desacoplamiento de Latencias:** El cálculo de la Densidad Espectral de Potencia (PSD) mediante FFT grandes ($N \geq 2048$) es una operación costosa y discontinua. Por el contrario, la demodulación de audio requiere un flujo continuo y determinista. Un bloqueo en el hilo de la FFT no debe interrumpir la continuidad del audio.
- **Eficiencia de CPU:** El uso de filtros IIR (*Biquads*) en lugar de filtros FIR extensos reduce drásticamente las operaciones de multiplicación-acumulación (MAC) por muestra, lo cual es vital para la arquitectura ARM de la Raspberry Pi.

- **Ancho de Banda de Red:** Transmitir PCM crudo (48 kHz, 16-bit) consume $\approx 1,5$ Mbps constantes. El sistema integra el códec **Opus**, reduciendo el ancho de banda a ≈ 32 –64 kbps, permitiendo la telemetría sobre enlaces LTE/4G precarios o inestables.

Para resolver el conflicto de latencias, se implementa una arquitectura de **Consumidor Dual** sobre un *Ring Buffer* (buffer circular) en memoria compartida, como se ilustra a continuación:

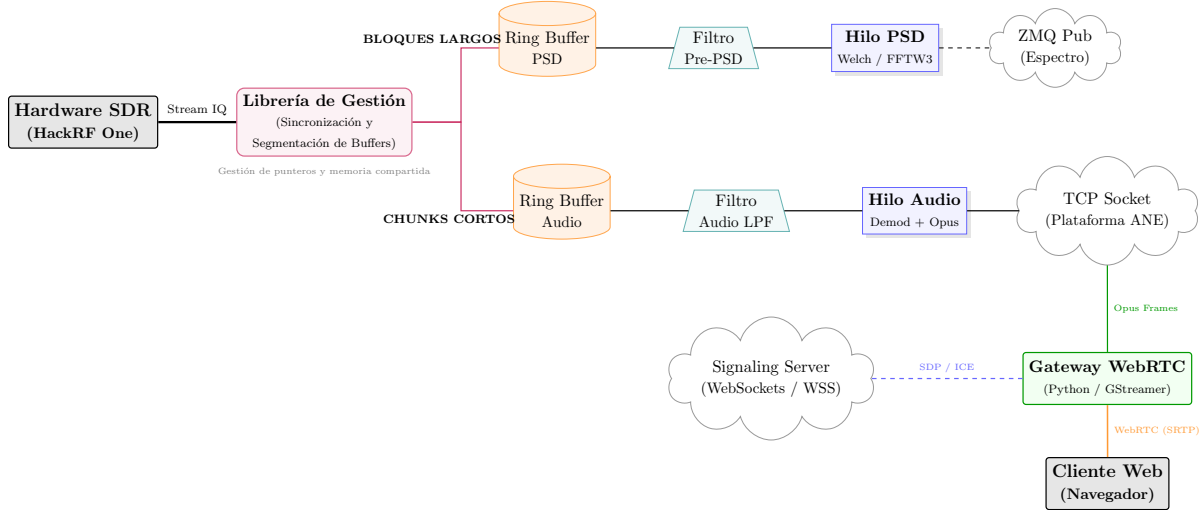


Figura 4: Arquitectura del sistema: Se detalla el desacoplamiento entre el motor de procesamiento en C y la capa de distribución WebRTC en Python.

3.2. Cadena de Demodulación y Métricas (AM/FM)

El hilo de audio ejecuta una cadena de procesamiento secuencial crítica. Dado que la señal de entrada proviene de un SDR de conversión directa, es imperativo eliminar la componente continua (DC) y filtrar el canal antes de proceder con la demodulación.

3.2.1. Preprocesamiento IQ

1. **Filtrado de Canal (*Channelizer*):** Se utiliza un filtro IIR Biquad pasabajos de orden 6 (Butterworth) aplicado independientemente a las ramas I y Q. Esto aísla el canal de interés (≈ 150 kHz para FM, ≈ 10 kHz para AM) del ruido de banda ancha, mejorando significativamente la relación señal a ruido (SNR) antes de la etapa no lineal de demodulación.

3.2.2. Algoritmos de Demodulación

El sistema soporta dos modos configurables dinámicamente:

A. Frecuencia Modulada (FM): Se emplea un discriminador de fase digital. La frecuencia instantánea es proporcional a la derivada de la fase. Computacionalmente, esto se aproxima calculando el ángulo entre muestras complejas consecutivas:

$$y[n] = \text{atan2}(\text{Im}(x[n] \cdot x^*[n-1]), \text{Re}(x[n] \cdot x^*[n-1])) \quad (5)$$

Posteriormente, se aplica un filtro de **deénfasis** ($75 \mu s$) para atenuar el ruido de alta frecuencia, conforme al estándar en radiodifusión comercial.

B. Amplitud Modulada (AM): Se utiliza un detector de envolvente asíncrono, calculado como la magnitud del vector IQ:

$$y[n] = \sqrt{I[n]^2 + Q[n]^2} \quad (6)$$

La señal resultante contiene una fuerte componente DC (la portadora) que debe ser eliminada para recuperar la información de audio.

3.2.3. Postprocesamiento y Métricas de Telemetría

Una innovación clave de este firmware es el cálculo de métricas de calidad de señal en banda (*piggybacked*) durante la demodulación, sin requerir pasadas adicionales sobre los datos:

- **DC Block (Bloqueo de Continua):** Filtro IIR de primer orden ($R = 0,995$) para eliminar el *offset* del oscilador local y la portadora AM.
- **Desviación FM (*Peak Deviation*):** Se detecta el pico máximo de frecuencia instantánea $|\Delta f|$ en ventanas de tiempo definidas.
- **Profundidad AM:** Se rastrean los valores mín./máx. de la envolvente para estimar el índice de modulación $m = \frac{Max-Min}{Max+Min}$.

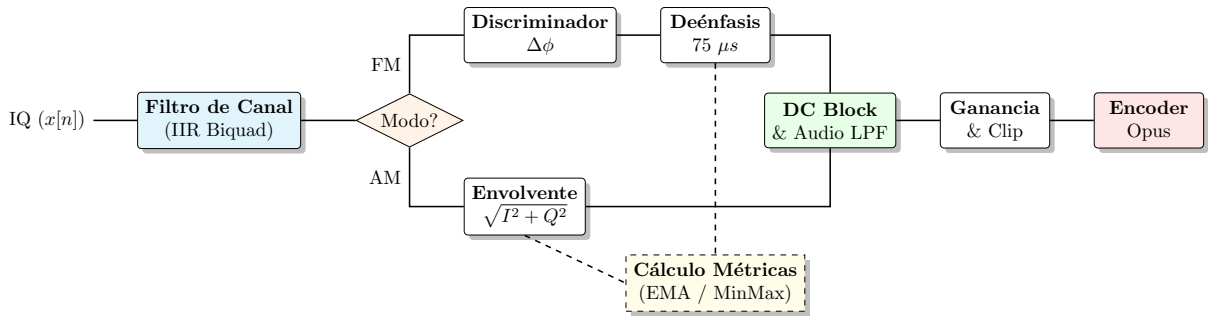


Figura 5: Diagrama de bloques detallado de la cadena de demodulación y extracción de métricas.

3.3. Métodos de Estimación Espectral de Alta Resolución

El motor permite conmutar entre dos métodos de estimación de la Densidad Espectral de Potencia (PSD). Ambos métodos buscan reducir la varianza del ruido, pero difieren en su capacidad para mitigar las fugas espectrales (*spectral leakage*).

3.3.1. Método de Welch (Periodograma Promediado)

El método de Welch mejora el periodograma estándar dividiendo la señal $x[n]$ en N_{blocks} segmentos solapados. Para cada bloque b , se aplica una ventana de apodización $w[m]$ (Hann o Hamming) de longitud M .

La potencia promediada se calcula como:

$$PSD_{avg}[u] = \frac{1}{N_{blocks}} \sum_{b=0}^{N_{blocks}-1} \frac{|X_b[u]|^2}{f_s \cdot \sum_{m=0}^{M-1} |w[m]|^2} \quad (7)$$

Donde $X_b[u]$ es la FFT del bloque ventaneado. El término del denominador garantiza la conservación de la energía (normalización por la potencia de la ventana y la frecuencia de muestreo f_s).

3.3.2. Banco de Filtros Polifásico (PFB - Polyphase Filter Bank)

A diferencia de Welch, el PFB utiliza un **filtro prototipo FIR** de longitud $L = M \times K$, donde M es el número de canales (puntos de la FFT) y K es el factor de superposición (*taps* por canal). Este método supera a la FFT directa al mejorar drásticamente la selectividad y reducir los lóbulos laterales.

1. Diseño del Filtro Prototipo: Se utiliza un filtro paso bajo $h[n]$ basado en una función Sinc truncada por una **ventana de Kaiser** para maximizar la atenuación del lóbulo lateral (≈ 80 dB con $\beta = 8,6$):

$$h[n] = \frac{\sin\left(\frac{\pi}{M}\left(n - \frac{L-1}{2}\right)\right)}{\pi\left(n - \frac{L-1}{2}\right)} \cdot \frac{I_0\left(\beta\sqrt{1 - \left(\frac{2n}{L-1} - 1\right)^2}\right)}{I_0(\beta)} \quad (8)$$

Para asegurar ganancia unitaria, el filtro se normaliza según: $h_{norm}[n] = h[n] / \sqrt{\sum |h[n]|^2}$.

2. Descomposición y Plegado (Folding): El filtro se descompone en una matriz polifásica $h_p[k, m] = h[k \cdot M + m]$. El proceso de "plegado" realiza la suma ponderada de la historia temporal de la señal sobre un buffer de longitud M , realizando un *aliasing* controlado en el dominio del tiempo:

$$y_{pre-fft}[m] = \sum_{k=0}^{K-1} x[n + kM] \cdot h_p[k, m] \quad (9)$$

3. Transformada y Normalización Física: Finalmente, se aplica la FFT al bloque plegado para obtener $Y[u]$. Para obtener unidades físicas de potencia (W/Hz), se escala según la tasa de muestreo f_s y el número de canales M :

$$PSD_{W/Hz}[u] = \frac{PSD_{avg}[u]}{f_s \cdot M} \quad (10)$$

3.3.3. Conversión a Unidades Logarítmicas (dBm)

Independientemente del método (Welch o PFB), el resultado final se convierte a una escala logarítmica para su visualización. Para comparar con un analizador de espectro comercial, se integra la potencia sobre el **Ancho de Banda de Resolución (RBW)** de un bin ($\Delta f = f_s/M$):

$$PSD_{dBm/bin}[u] = 10 \log_{10}(PSD_{W/Hz}[u]) + 30 + 10 \log_{10}\left(\frac{f_s}{M}\right) \quad (11)$$

Esta normalización asegura que el nivel de potencia reportado sea independiente del tamaño de la FFT elegida, permitiendo mediciones consistentes bajo diferentes configuraciones de resolución.

3.4. Corrección de Respuesta (Blind Equalization)

Para compensar la coloración del ruido introducida por el hardware (filtros analógicos y amplificadores), el motor implementa un algoritmo de ecualización ciega en tres pasos:

1. **Estimación de Tendencia:** $T[u] = \text{mediana}(PSD[u-w, \dots, u+w])$. Se aplica un filtro de mediana para extraer la forma del piso de ruido ignorando picos de señal.
2. **Vector de Corrección:** $C[u] = \mu - T[u]$, donde μ es la mediana global del espectro.
3. **Espectro Final:** $PSD_{final}[u] = PSD_{dBm/bin}[u] + C[u]$.

4. Postprocesamiento Espectral para Cumplimiento Normativo

Esta sección explica, de forma **intuitiva y operativa**, cómo el sistema transforma una traza PSD (un arreglo de potencia vs. frecuencia) en un **dictamen de cumplimiento normativo** frente a una base de licencias. La intención es que cualquier lector pueda responder:

- **Qué se detecta:** dónde hay emisiones reales (y no ruido).
- **Qué se mide:** f_c , ancho de banda BW y potencia integrada P .
- **Cómo se decide cumplimiento:** cómo se asocia cada emisión a una licencia (por DANE) y qué reglas hacen que una emisión *cumpla* o *no cumpla*.

4.1. Idea clave: una PSD no “dice” sola si hay infracción

Una PSD es una fotografía estadística del espectro. En la práctica:

- El **piso de ruido** no es constante: cambia con ganancia, temperatura, banda, interferencias, y variación del receptor.
- Hay **picos falsos** (ruido impulsivo, espurios del SDR, variación de la FFT, DC residual).
- Las reglas normativas no se basan solo en el pico: normalmente exigen comparar **frecuencia asignada, ancho permitido y potencia máxima**.

Por eso el postprocesamiento se diseña como una cadena de decisiones con trazabilidad (explicar por qué se marcó “cumple/no cumple”).

4.2. Diagrama de flujo del módulo de cumplimiento

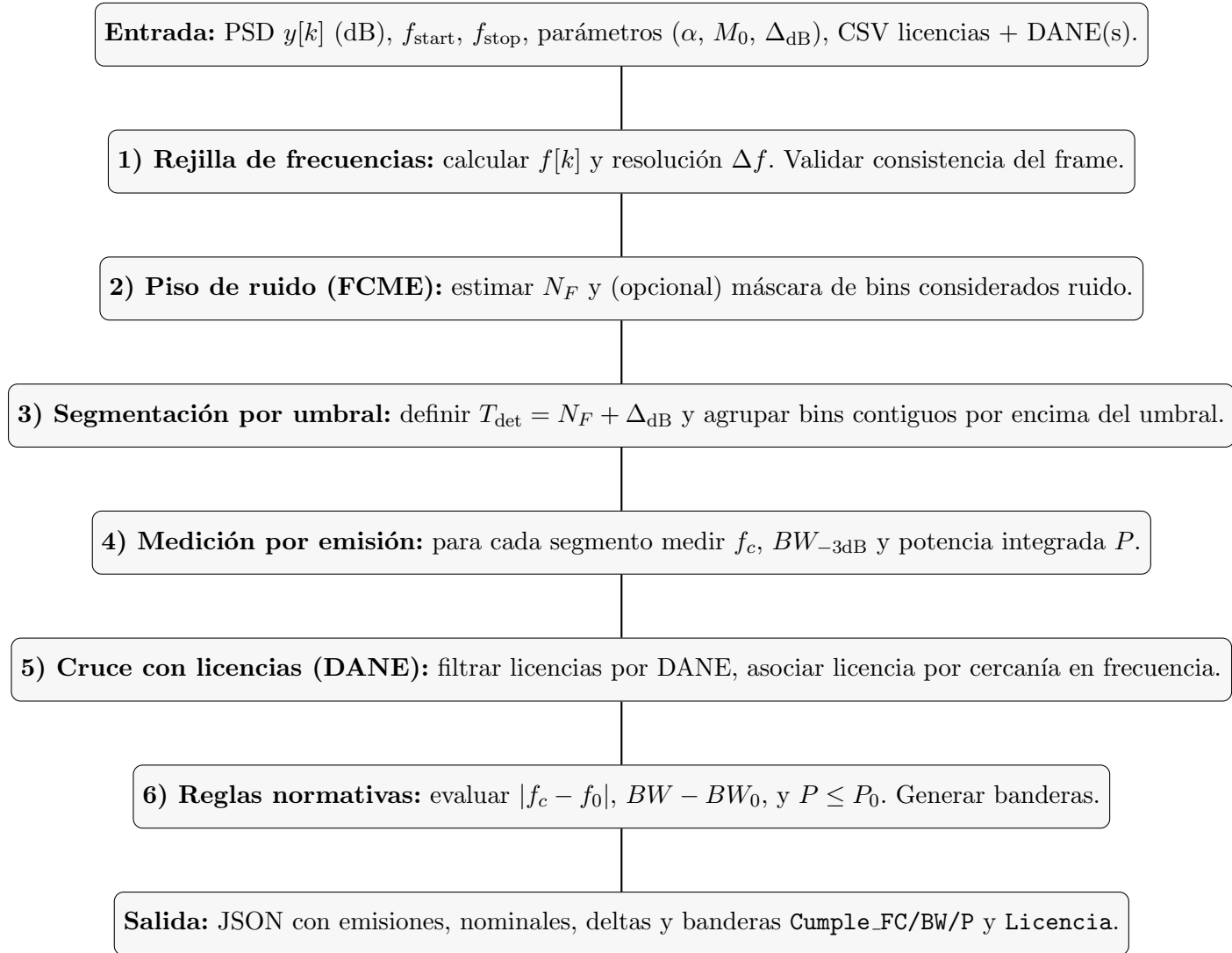


Figura 6: Flujo de postprocesamiento orientado a cumplimiento normativo.

4.3. Definiciones y variables (para leer el resto sin perderse)

Símbolo Campo	/ Significado práctico
$y[k]$	Valor de la PSD en el bin k , en dB (típicamente dBm, o densidad dBm/Hz según calibración).
N	Número de bins del espectro.
$f_{\text{start}}, f_{\text{stop}}$	Frecuencia inicial y final del frame.
Δf	Resolución (Hz/bin): qué tan fino es el “paso” del eje de frecuencia.
N_F	Piso de ruido estimado (dB). Punto de referencia para decidir qué sobresale del ruido.
α	Parámetro FCME: control de aceptación de bins como ruido.
M_0	Tamaño inicial (bins) del conjunto de ruido en FCME.
Δ_{dB}	Margen por encima del ruido para declarar detección (control de falsos positivos/negativos).
$\mathcal{B}$	Conjunto de bins pertenecientes a una emisión (segmento contiguo por encima del umbral).
f_c, BW, P	Parámetros medidos: frecuencia central, ancho (a -3 dB) y potencia integrada.
f_0, BW_0, P_0	Parámetros nominales (licencia) contra los cuales se compara.

Tabla 2: Glosario operativo del postprocesamiento.

4.4. Entrada: qué necesita el algoritmo y por qué

El postprocesamiento de cumplimiento necesita dos entradas diferentes:

1. **Medición espectral:** el frame con $y[k]$ y el rango $[f_{\text{start}}, f_{\text{stop}}]$. Sin estos datos no hay forma de ubicar emisiones en frecuencia ni de integrar potencia.
2. **Marco autorizatorio (licencias):** un CSV con licencias, incluyendo el **código DANE** del área de interés. Sin esto, el sistema podría detectar emisiones, pero no puede afirmar si están autorizadas.

4.5. Rejilla de frecuencias: por qué se calcula y cómo afecta todo

A partir del rango y el número de bins:

$$f[k] = f_{\text{start}} + k \Delta f, \quad \Delta f \approx \frac{f_{\text{stop}} - f_{\text{start}}}{N - 1}. \quad (12)$$

Intuición. La PSD es un vector sin significado si no sabemos qué frecuencia corresponde a cada posición. Además:

- Δf define la **precisión** con la que podemos reportar f_c . Si Δf es grande, f_c quedará “cuantizado”.

- Δf define la **granularidad** del ancho de banda. Si una emisión es angosta y Δf es grande, BW puede medirse con error.
- Δf define cómo se integra potencia cuando el eje vertical representa densidad.

4.6. Estimación del piso de ruido mediante FCME (qué hace y por qué es apropiado)

4.6.1. Problema que resuelve

Para detectar emisiones, primero debemos saber **qué es “ruido”** en esa captura. Un promedio simple de $y[k]$ suele ser malo porque:

- Las señales elevan la media y hacen que el umbral se vuelva demasiado alto.
- Un umbral alto causa falsos negativos (señales reales ignoradas).

FCME se contempla porque construye el ruido desde los bins **más bajos** y va aceptando bins de forma controlada, evitando que señales dominen el estimador.

4.6.2. Conversión a escala lineal: por qué se hace

FCME usa medias y umbrales multiplicativos. Eso se vuelve natural en potencia lineal:

$$X(k) = 10^{y_{\text{dB}}(k)/10}. \quad (13)$$

Intuición. En dB, sumar y promediar no representa correctamente la física de la potencia. En lineal sí: la potencia total es suma de potencias.

4.6.3. Parámetros FCME (M_0 y α) con explicación intuitiva

- M_0 : arranque del conjunto de ruido. Si el sistema inicia con pocos bins, la media puede ser inestable; si inicia con demasiados bins, podría incluir partes de señal. Por eso M_0 se elige como fracción de N con un mínimo razonable.
- α : “factor de tolerancia”. Imagina que el ruido fluctúa: algunos bins quedan un poco más altos. α define cuánto más alto que la media del ruido puede ser un bin y aún considerarse ruido.

4.6.4. FCME paso a paso (con lectura humana)

1) Ordenar de menor a mayor.

$$X(1) \leq X(2) \leq \dots \leq X(N). \quad (14)$$

Intuición. Los bins más bajos casi siempre son ruido. Si empezamos por ellos, tenemos una base “limpia”.

2) Tomar los primeros M_0 como ruido inicial.

$$\mathcal{N}^{(0)} = \{X(1), X(2), \dots, X(M_0)\}. \quad (15)$$

3) Calcular la media del ruido inicial.

$$\mu^{(0)} = \frac{1}{M_0} \sum_{i=1}^{M_0} X(i). \quad (16)$$

4) Construir el umbral FCME.

$$T^{(0)} = \alpha \mu^{(0)}. \quad (17)$$

Intuición. Si un bin está por debajo de $\alpha\mu$, es compatible con ser ruido; si supera eso, probablemente ya no es ruido.

5) **Regla *Forward Consecutive*:** ir agregando bins uno por uno.

$$\text{Si } X(M+1) \leq \alpha\mu \Rightarrow \begin{cases} M \leftarrow M+1, \\ \mu \leftarrow \frac{1}{M} \sum_{i=1}^M X(i). \end{cases}$$

El proceso se detiene cuando $X(M+1) > \alpha\mu$.

6) **Piso de ruido final en dB.**

$$\widehat{\sigma_n^2} = \mu, \quad N_F = 10 \log_{10}(\widehat{\sigma_n^2}). \quad (18)$$

Qué te entrega FCME (en la práctica). No solo entrega N_F ; también define de forma implícita *cuáles bins fueron considerados ruido*. Eso es valioso para auditoría: permite explicar por qué el umbral quedó en cierto nivel.

4.7. Del piso de ruido al umbral de detección: controlar falsos positivos y negativos

Una vez se estima N_F , el detector no usa N_F directamente como umbral, sino:

$$T_{\text{det}} = N_F + \Delta_{\text{dB}}. \quad (19)$$

4.7.1. Por qué existe Δ_{dB}

Si usáramos $T_{\text{det}} = N_F$, el ruido mismo cruzaría el umbral por fluctuaciones naturales (ruido impulsivo o variación estadística). Δ_{dB} actúa como un “colchón”:

- Δ_{dB} pequeño: más sensibilidad, pero más falsos positivos.
- Δ_{dB} grande: menos falsos positivos, pero riesgo de perder señales débiles.

Intuición. Es el control principal para ajustar el comportamiento del sistema según el objetivo de la campaña.

4.8. Detección de emisiones: segmentación (cómo se define una emisión)

4.8.1. Segmentación por contigüidad

Se considera emisión candidata cualquier **grupo contiguo** de bins que cumpla $y[k] \geq T_{\text{det}}$. Formalmente, cada emisión es un conjunto $\mathcal{B} \subset \{0, \dots, N-1\}$ contiguo.

Por qué segmentar y no solo buscar máximos. El cumplimiento no depende solo del máximo: para medir BW y potencia se necesita una ventana asociada a la emisión. La segmentación proporciona esa ventana.

4.8.2. Filtros de robustez (para evitar artefactos)

En espectros reales pueden aparecer detecciones espurias. Por eso se contemplan criterios mínimos:

- **Longitud mínima del segmento:** evita que un bin aislado (ruido) se reporte como emisión.
- **Consistencia local:** si el segmento es extremadamente irregular, puede corresponder a artefactos o a una mala estimación del umbral.

Intuición. En un contexto normativo es preferible ser conservador antes que reportar emisiones fantasma.

4.9. Medición por emisión: f_c , BW y potencia (y por qué se contemplan)

Estas tres métricas se contemplan porque son las que típicamente aparecen en licencias: frecuencia autorizada, ancho permitido y potencia máxima.

4.9.1. Frecuencia central f_c (cómo se define y qué significa)

Dentro del segmento $\mathcal{B}$, se toma el bin de mayor amplitud:

$$k_{\text{pico}} = \arg \max_{k \in \mathcal{B}} y[k], \quad f_c = f[k_{\text{pico}}]. \quad (20)$$

Intuición. Para muchas emisiones (portadoras, pilotos, tonos dominantes), el máximo es el punto más estable para referenciar la frecuencia y compararla con f_0 .

4.9.2. Ancho de banda $BW_{-3\text{dB}}$ (qué mide realmente)

Se construye un umbral relativo al pico:

$$y_{\text{thr}} = y[k_{\text{pico}}] - 3 \text{ dB}. \quad (21)$$

Luego se busca hacia la izquierda y derecha el cruce con y_{thr} (interpolando entre bins para evitar cuantización), y:

$$BW_{-3\text{dB}} = f_{\text{right}} - f_{\text{left}}. \quad (22)$$

Intuición. Es una medida sencilla del “cuerpo principal” de la emisión alrededor del máximo. No pretende sumar toda la energía, sino dar una medida comparable entre capturas.

4.9.3. Potencia integrada P (por qué no basta el pico)

Una licencia suele limitar potencia total (o potencia en canal), no el pico máximo. Dos emisiones con el mismo pico pueden tener potencias muy diferentes si una es más ancha. Por eso se integra potencia en la ventana de la emisión:

$$P_W \approx \sum_{k \in \mathcal{B}} 10^{y[k]/10} \Delta f, \quad (23)$$

$$P_{\text{dBm}} = 10 \log_{10}(P_W) + 30. \quad (24)$$

Aclaración crítica sobre unidades (para evitar errores de interpretación). Dependiendo del generador de PSD:

- Si $y[k]$ es **densidad** (dBm/Hz), entonces multiplicar por Δf es correcto para pasar a potencia por bin.
- Si $y[k]$ ya es **potencia por bin** (dBm/bin), entonces no se debe volver a escalar por Δf .

En implementación, mantener Δf explícito permite documentar el supuesto y evitar reportes inflados o subestimados.

4.10. Cruce con licencias por DANE: por qué se hace así

4.10.1. Por qué se filtra por DANE

El código DANE amarra el análisis a un territorio. Una misma frecuencia puede estar licenciada en un municipio y no en otro. Si se comparara contra licencias de todo el país, el sistema podría asociar incorrectamente una emisión a una licencia de otra región, generando falsos *cumple*.

4.10.2. Cómo se asocia una emisión a una licencia (criterio principal: frecuencia)

Dada una emisión (f_c, BW, P) :

1. Se filtra el CSV por el/los DANE configurados.
2. Se seleccionan licencias candidatas cuya frecuencia nominal f_0 sea cercana a f_c .
3. Se elige la licencia con menor $|f_c - f_0|$ dentro de tolerancia. En casos ambiguos, se puede usar como apoyo BW y potencia nominal.

Intuición. En cumplimiento, el primer “ancla” es la frecuencia: una licencia está asignada a una frecuencia nominal. Luego, ancho y potencia refinan el veredicto.

4.11. Reglas de cumplimiento normativo (qué se evalúa y cómo se interpreta)

Una vez asociada la licencia nominal (f_0, BW_0, P_0) , se evalúan tres condiciones:

- **Frecuencia central:** $|f_c - f_0| \leq 0,1 \text{ MHz}$.
- **Ancho de banda:** $(BW - BW_0) \leq 10 \text{ kHz}$.
- **Potencia:** $P \leq P_0$.

4.11.1. Por qué se contemplan tolerancias

Ninguna medición es perfecta:

- La FFT tiene resolución finita (Δf) .
- Puede existir corrimiento de LO o drift del oscilador.
- El BW medido depende de ventana, promediado (Welch) y condiciones de la señal.

Las tolerancias evitan que pequeñas variaciones técnicas se traduzcan en falsos “no cumple”. A la vez, la condición de potencia protege el límite autorizador: si la potencia integrada supera P_0 , se debe reportar no conformidad.

4.11.2. Banderas de salida: hacer el resultado explicable

El sistema entrega banderas por emisión:

- **Cumple_FC:** dentro de tolerancia de frecuencia.
- **Cumple_BW:** ancho no excede lo permitido (con margen).
- **Cumple_P:** potencia no excede el máximo.
- **Licencia:** true solo si todas las anteriores son true.

Intuición. No basta decir “no cumple”: hay que decir *qué falló*. Eso hace el sistema auditable y útil para acciones correctivas (calibración, ajuste de umbral, verificación de licencia, etc.).

4.12. Salida estructurada (JSON): qué contiene y por qué

La salida se emite en JSON para integrarse con el plano de control, reportes y trazabilidad. Contiene:

- **Metadatos del frame:** f_{start} , f_{stop} , N , Δf , y metadatos opcionales (timestamp/-MAC).
- **Emisiones medidas:** por cada emisión, $\{f_c, BW, P\}$.
- **Cruce con licencia:** nominales $\{f_0, BW_0, P_0\}$ (si aplica) y deltas informativos.
- **Cumplimiento:** `Cumple_FC`, `Cumple_BW`, `Cumple_P`, `Licencia`.

Por qué se contempla trazabilidad. En contexto normativo hay que poder reconstruir decisiones: qué se midió, bajo qué parámetros, contra qué licencia, y por qué se concluyó cumplimiento o no. Por eso se reportan parámetros, nominales y banderas, y se preservan metadatos.

5. Anexo Metrológico: Caracterización y Validación Estadística del Sistema ANE-HX

Esta sección detalla los procedimientos técnicos, fundamentos matemáticos y protocolos de validación estadística empleados para certificar el rendimiento del sistema de monitoreo de espectro ANE-HX. Para garantizar la trazabilidad y representatividad de los resultados, cada métrica se basa en la adquisición masiva de datos ($N = 10,000$ muestras por punto de prueba) y el uso de análisis de varianza (ANOVA) para la validación inter-dispositivo.

5.1. Determinación de la Sensibilidad y Caracterización del DANL

El Displayed Average Noise Level (DANL) representa la densidad espectral de potencia del ruido térmico intrínseco del sistema. Se modela como un proceso estocástico estacionario y ergódico.

1. **Fundamento Matemático:** La potencia de ruido medida P_N en un ancho de banda de resolución (RBW) se normaliza a un ancho de banda unitario de 1 Hz para obtener la densidad espectral:

$$DANL(f)_{dBm/Hz} = 10 \log_{10} \left(\frac{1}{N} \sum_{i=1}^N P_{i,Watts} \right) - 10 \log_{10}(RBW) \quad (25)$$

2. **Procedimiento Experimental:** Se termina la entrada de RF en una carga de referencia de 50Ω . Se realizan $N = 10,000$ barridos espectrales para cada versión del sensor (ANE1-ANE10).
3. **Validación Estadística:** Se calcula la media poblacional (μ) y la desviación estándar (σ) del ruido. La desviación estándar permite cuantificar la estabilidad del front-end. Se aplica un **ANOVA de un factor** para comparar las medias de DANL entre las 10 unidades. La hipótesis nula $H_0 : \mu_1 = \mu_2 = \dots = \mu_{10}$ se evalúa con un nivel de significancia $\alpha = 0,05$. Un valor $p > 0,05$ garantiza que el proceso de fabricación y blindaje de los sensores es estadísticamente uniforme.

5.2. Exactitud de Frecuencia y Estabilidad del Oscilador Local

La precisión en la sintonía es crítica para la detección de portadoras de banda estrecha. Se evalúa el desplazamiento (offset) del oscilador local respecto a un patrón de frecuencia atómico.

1. **Justificación de la Métrica PPM:** Dado que el error absoluto en Hz escala con la frecuencia, se utiliza la normalización en Partes por Millón (PPM) para obtener una medida de estabilidad intrínseca del TCXO:

$$Error_{PPM} = \left(\frac{f_{medida} - f_{referencia}}{f_{referencia}} \right) \times 10^6 \quad (26)$$

2. **Tratamiento de Datos:** Se inyecta un tono puro y se registran 10,000 estimaciones de la frecuencia central. El uso de esta población masiva permite que el error de estimación de la media sea despreciable ($\sigma_{\bar{x}} = \sigma/\sqrt{10000}$), cumpliendo con el Teorema del Límite Central.
3. **ANOVA de Estabilidad:** Se realiza un ANOVA para determinar si el error PPM es dependiente de la frecuencia de operación (barrido de 1 MHz a 6 GHz). Si el estadístico F no es significativo, se valida que el error es puramente sistemático y puede ser compensado mediante una constante de corrección única en el firmware.

5.3. Error de Potencia y Protocolo de Calibración Metrológica

Para corregir la no linealidad de la respuesta en frecuencia del receptor, se realiza una transferencia de patrón utilizando un analizador de espectro Keysight N9000B como referencia trazable.

1. **Derivación del Bias (Sesgo):** El error sistemático de potencia $\epsilon(f)$ se estima promediando 10,000 muestras de la diferencia entre el sensor ANE y el equipo de referencia:

$$\epsilon(f) = \frac{1}{10000} \sum_{i=1}^{10000} (P_{ANE,i} - P_{Ref,i}) \quad (27)$$

2. **Función de Calibración:** La corrección se aplica sumando el inverso del sesgo medido, ajustado por un factor de incertidumbre tipo A:

$$P_{calibrada} = P_{medida} - \epsilon(f) \quad (28)$$

3. **Validación ANOVA de Calibración:** Este es el procedimiento de cierre metrológico. Se comparan tres poblaciones de 10,000 datos cada una: *Referencia*, *Sensor Crudo* y *Sensor Calibrado*. El ANOVA entre la *Referencia* y el *Sensor Calibrado* debe arrojar un valor $p \rightarrow 1$ y una diferencia de medias cercana a 0 dB. Esto certifica que el sistema ANE-HX es estadísticamente indistinguible de un instrumento de laboratorio en términos de precisión de amplitud.

5.4. Linealidad y Rango Dinámico Operativo

El rango dinámico define los límites inferior (piso de ruido) y superior (compresión) del sistema.

1. **Estimación Robusta por Regresión:** Se utiliza una regresión lineal sobre 10,000 puntos de potencia creciente. El modelo de respuesta lineal es $P_{out} = m \cdot P_{in} + b$. Se minimiza la función de costo:

$$J(m, b) = \sqrt{\frac{1}{N} \sum (P_{out,i} - (m \cdot P_{in,i} + b))^2 + \lambda |m - 1|} \quad (29)$$

Donde λ penaliza desviaciones de la pendiente unitaria ideal.

2. **Análisis de Residuos (RMSE):** La desviación estándar de los residuos de la regresión (RMSE) justifica la calidad del ajuste. Un $RMSE < 1$ dB en todo el rango dinámico asegura que el sistema no introduce distorsión no lineal significativa.

3. **Prueba t de Student sobre la Pendiente:** Para cada unidad ANE, se realiza una prueba t para verificar la hipótesis $H_0 : m = 1$. La validación de esta hipótesis es el fundamento estadístico para garantizar que el rango dinámico reportado es lineal y confiable para mediciones de densidad espectral de potencia (PSD).

Este marco estadístico y matemático garantiza que el sistema ANE-HX no solo cumple con las especificaciones técnicas, sino que posee la estabilidad metrológica requerida para aplicaciones de supervisión del espectro radioeléctrico a nivel institucional.

6. Interfaz de Control y Configuración

6.1. Estructura de Comandos JSON

Campo JSON	Tipo	Descripción
<code>rf_mode</code>	string	Modo de operación (realtime, campaign, fm).
<code>center_freq_hz</code>	uint64	Frecuencia central de sintonía en Hz.
<code>sample_rate_hz</code>	double	Frecuencia de muestreo (IQ Rate).
<code>rbw_hz</code>	int	Resolución de ancho de banda deseada.

Tabla 3: Parámetros de configuración del sistema.

7. Testing (Pruebas Automatizadas)

7.1. Propósito

Se implementó una batería de pruebas automatizadas con **pytest** para reducir regresiones y aumentar la confiabilidad del sistema. La suite valida comportamiento correcto en rutas normales (*happy paths*), manejo de condiciones de borde (*edge cases*) y tolerancia a fallos típicos de entorno (E/S, logging, IPC y fallas simuladas de red), priorizando **determinismo** y **reproducibilidad**.

7.2. Alcance y limitaciones

El alcance se concentra en el código **Python** del repositorio, que contiene la lógica de orquestación y utilidades operativas:

- `cfg.py`: logging, captura de `stdout/stderr`, rotación/limpieza de logs, utilidades de tiempo y ejecución controlada.
- `campaign_runner.py`, `orchestrator.py` y módulos de soporte (`retry_queue.py`, `init_sys.py`, `kal_sync.py`, `status.py`).
- `utils/`: utilidades transversales de requests, controladores de IPC (ZMQ), validación de configuración y utilidades de E/S y estado.
- **Infraestructura de pruebas** (`tests/`): `conftest.py` centraliza fixtures y ajustes de `sys.path` para asegurar resolución correcta de imports; además se incluyen utilidades de soporte bajo `tests/tools` cuando se requiere aislar dependencias.

Quedan fuera de alcance, por diseño, las validaciones **end-to-end** con SDR real (HackRF/USRP), pruebas prolongadas bajo carga de captura IQ continua y pruebas completas del motor nativo en C/FFTW (si aplica). Estas deben cubrirse con pruebas de integración y rendimiento en un entorno dedicado.

7.3. Entorno de ejecución

Las pruebas se ejecutan en Linux/WSL con **Python 3.12** dentro de un entorno virtual (`.venv`). Se emplean: `pytest`, `pytest-cov`, `pytest-asyncio` (modo *strict*) y `pytest-mock`. Se recomienda ejecutar desde la raíz del repositorio para garantizar resolución correcta de imports (módulos como `cfg`, `functions`, `retry_queue`, `orchestrator` y `utils`). En esta iteración, se validó explícitamente la colección completa y la ausencia de pruebas omitidas mediante `-ra` y `--collect-only`.

7.4. Comandos de ejecución

```
1 cd ~/pytest/Origin/SDR-SpectrumMonitoring-Sensor
2 find . -type d -name "__pycache__" -prune -exec rm -rf {} \;
3 rm -rf .pytest_cache
4
5 # Ejecutar suite completa
6 pytest -q
```

Listing 1: Ejecución limpia y verificación completa (WSL/Linux)

```
1 # 1) Mostrar resumen si existiera algo omitido
2 pytest -ra
3
4 # 2) Contar tests colectados
5 pytest --collect-only -q | grep -E '::' | wc -l
6
7 # 3) Confirmación final con reporte (si hubiera skips/xfail,
8     aqu apareceran)
9 pytest -q -ra
```

Listing 2: Resumen detallado (skips/xfail/xpass) y verificación de colección

```
1 pytest -q --cov=. --cov-report=term-missing
```

Listing 3: Cobertura global del repositorio

```
1 pytest -q --cov=cfg --cov-report=term-missing
```

Listing 4: Cobertura dirigida a un módulo crítico

7.5. Estrategia de pruebas

La suite se construyó con énfasis en:

- **Aislamiento de efectos secundarios:** uso de `tmp_path` para E/S temporal y *monkeypatching* para controlar tiempo, rutas, entorno y dependencias externas.

- **Simulación de fallos controlados:** pruebas específicas inducen errores (p. ej., respuestas HTTP no-200, JSON inválido, errores de serialización, fallos de socket/IPC, fallos de escritura/borrado) para confirmar que el sistema falla de manera segura.
- **Cobertura de bordes:** se ejercitan ramas de manejo de excepciones y condiciones límite (logging/flush, rotación de logs, limpieza por patrón, validación estricta de configuración, etc.).
- **Pruebas asíncronas deterministas:** uso de `pytest-asyncio` en modo *strict* y timers controlados para evitar flakiness en la lógica en tiempo real.
- **Pruebas de humo (*smoke tests*):** validación rápida de imports y dependencias para detectar problemas tempranos de entorno.

7.6. Resultados

La suite alcanzó un estado estable y reproducible:

- Resultado: **168 pruebas** ejecutadas con éxito (**168 passed**).
- Verificación de completitud: `pytest -ra` no reportó *skips*, *xfail* ni *xpass*; además, `--collect-only` confirmó la colección de **168** casos.

7.7. Conclusiones

1. La batería de pruebas reduce el riesgo de regresiones y estabiliza el mantenimiento del sistema, especialmente en módulos operativos críticos (orquestración, requests, IPC y estado).
2. El uso de `-ra` y `--collect-only` añade una verificación explícita de que no existen pruebas omitidas (*skipped*) ni comportamientos esperados-fallidos (*xfail*), fortaleciendo la trazabilidad del resultado final.
3. Se recomienda complementar esta suite unitaria con pruebas de integración (ZMQ/API/SDR real) y pruebas de rendimiento en un entorno controlado, manteniendo separadas ambas capas de validación.

8. Infraestructura de Gestión y Aprovisionamiento Remoto OTA-ANE v7

La Agencia Nacional del Espectro (ANE) utiliza el sistema **OTA v7** como una capa de orquestración centralizada. Este sistema permite la administración de sensores distribuidos geográficamente mediante un canal seguro, garantizando que el acceso al plano de control del sensor solo sea posible a través de una red privada cifrada.

8.1. Justificación Técnica del Diseño

El diseño se fundamenta en tres pilares de seguridad y eficiencia:

- **Criptografía de Alto Rendimiento (WireGuard):** A diferencia de VPNs basadas en SSL/TLS, WireGuard opera en el espacio del kernel con el protocolo UDP 2201, lo que reduce la sobrecarga (overhead) en la Raspberry Pi, permitiendo que el procesamiento DSP no se vea afectado por la latencia del túnel.
- **Defensa Perimetral Activa (Port Knocking):** El puerto de aprovisionamiento (2204) permanece en estado *stealth* (invisible) hasta que el sensor ejecuta una secuencia de "golpes" en los puertos TCP 2203, 2205, 2207 y 2209. Esto evita ataques de denegación de servicio (DoS) y escaneos de vulnerabilidades sobre el HUB.
- **Persistencia de Identidad por MAC:** Se utiliza el direccionamiento estático vinculado a la dirección física (MAC) del sensor. Esto garantiza que, ante un reinicio o fallo de red, el sensor siempre recupere la misma IP privada, facilitando la recolección de datos histórica.

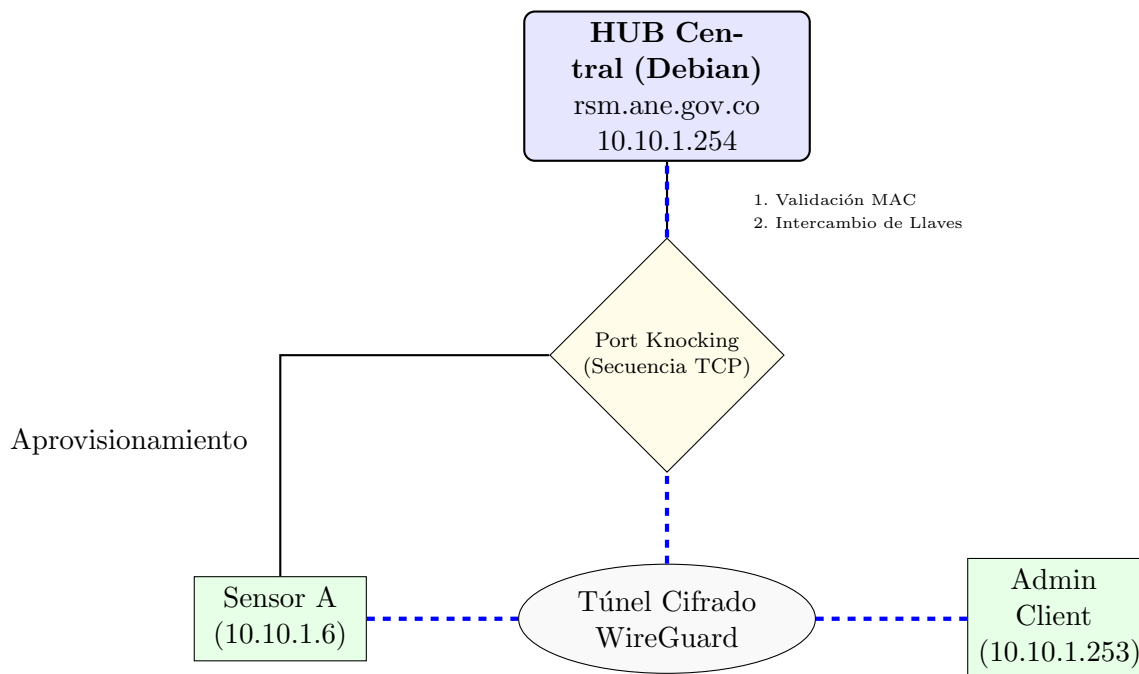


Figura 7: Diagrama de Flujo del Sistema de Aprovisionamiento y Red Privada OTA-ANE.

8.2. Implementación y Despliegue del Sistema

8.2.1. Configuración del Servidor (HUB)

El despliegue en el servidor Debian 12 requiere la preparación del archivo `sensors.csv` en la ruta `/opt/provisioner/`. Este archivo actúa como una lista blanca (Whitelist):

1. **Registro:** Se añade la MAC del dispositivo y la IP fija deseada (e.g., ANE_N1,d8:3a:dd:f7:1a:cc,
2. **Instalación:** Se ejecuta el script maestro `sudo ./ota-hub-install`, el cual aprovisiona las interfaces `wg0` y activa el servicio `provisioner`.

8.2.2. Aprovisionamiento del Sensor (Bootstrap)

Para un sensor nuevo o reinstalado, el proceso es totalmente automático mediante un *script de bootstrap* que realiza el *port knocking* y configura la VPN automáticamente:

```
1 curl -fsSL http://rsm.ane.gov.co:2204/bootstrap_provision.sh |  
    sudo bash
```

Listing 5: Comando de aprovisionamiento automático

8.3. Gestión Administrativa y Diagnóstico

El acceso remoto a los sensores se realiza desde el **Ciente Administrativo** (Windows/Linux) utilizando el archivo de configuración `mgmt-client.conf`. Una vez activo el túnel, el administrador puede acceder por SSH a cualquier sensor dentro de la red 10.10.1.0/24.

Comandos de Verificación Crítica:

- `sudo wg show`: Verifica qué sensores tienen un túnel activo y la cantidad de datos transferidos.
- `sudo journalctl -u provisioner -f`: Permite auditar en tiempo real si un sensor está siendo rechazado por una dirección MAC no autorizada.

8.4. Mantenimiento y Errores Comunes

- **Error: “mac not authorized”**: El sensor está intentando conectarse con una MAC distinta a la registrada (común si se cambia de Ethernet a Wi-Fi). Se debe actualizar `sensors.csv` y reiniciar el servicio: `sudo systemctl restart provisioner`.
- **Re-enrolamiento**: Si un sensor pierde su configuración, ejecutar nuevamente el comando de *bootstrap*. El HUB reconocerá la MAC y reasignará la misma IP y llaves de cifrado.

9. Lógica de Plataforma

La plataforma **ANE — Sensado Espectral** opera bajo una arquitectura de procesamiento distribuido que garantiza la integridad de los datos técnicos y la disponibilidad continua del hardware. La lógica del sistema se aleja de los modelos convencionales de escritorio para implementar un ecosistema de *Edge Computing*, donde el procesamiento pesado de señales ocurre en el sensor, mientras que la analítica y visualización se gestionan mediante una interfaz web de alta fidelidad.

9.1. Arquitectura Funcional y Flujo de Valor

El diseño lógico del sistema está optimizado para resolver el conflicto entre el procesamiento de datos a alta velocidad (≈ 40 MB/s de datos crudos) y la visualización remota en entornos con ancho de banda variable. Esta eficiencia se logra mediante una estructura de tres niveles operativos:

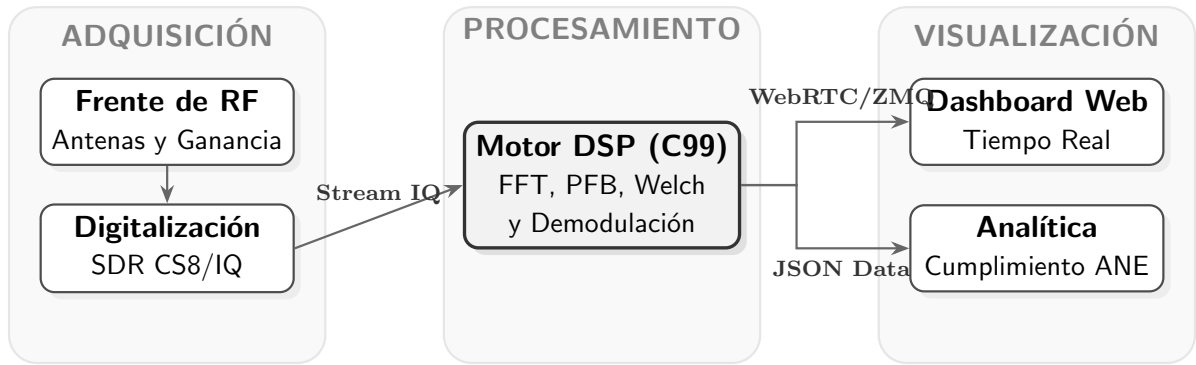


Figura 8: Arquitectura lógica del flujo de datos: desde la captura de RF hasta la entrega de información técnica.

9.2. Lógica de Monitoreo y Salud del Hardware

Para garantizar que el hardware SDR permanezca calibrado y libre de sobrecalentamientos durante jornadas intensas de monitoreo, la plataforma implementa una lógica de **Gestión de Recursos Críticos**:

- **Temporizador de Sesión Dinámico:** Cada sesión de monitoreo en tiempo real cuenta con un límite automático de 10 minutos. Esta lógica asegura que el hardware sea liberado para otros usuarios o campañas programadas, evitando el acaparamiento del recurso y el estrés térmico del nodo.
- **Compensación Automática de Artefactos:** El sistema detecta y elimina de forma nativa el *DC Spike* (pico central) mediante el algoritmo *Offset & Crop*. Esto entrega al usuario una traza limpia donde cada pico visualizado corresponde a una emisión real y no a una imperfección del receptor.
- **Telemetría Predictiva:** El sensor reporta en tiempo real su carga de CPU y temperatura. Si el nodo detecta un estado crítico, la lógica de la plataforma prioriza el enfriamiento y la salvaguarda de datos antes que la visualización.

9.3. Automatización y Rigor en las Campañas

La lógica de ejecución de campañas se basa en el principio de **Persistencia Segura**. A diferencia del monitoreo manual, las campañas activan un modo de operación donde el almacenamiento local (`/dev/shm` y `SSD`) se gestiona mediante escrituras atómicas.

El proceso de cumplimiento normativo se ejecuta de forma asíncrona: una vez capturada la información espectral, el motor de post-procesamiento cruza los datos con la base de licencias *DANE*. El sistema no solo identifica la frecuencia central (f_c), sino que integra la potencia total en el canal y mide el ancho de banda a -3 dB, permitiendo generar el **Reporte de Cumplimiento** sin intervención humana. Esta lógica elimina la subjetividad en la interpretación de los límites de potencia y garantiza la trazabilidad técnica necesaria para procesos administrativos.

9.4. Continuidad de Operación (Redes Críticas)

El sistema está diseñado para operar en condiciones de red inestables (enlaces 4G/LTE). La lógica de comunicación implementa una **Cola de Reintentos (FIFO)**: si el

sensor pierde conectividad con la plataforma central, los datos recolectados se almacenan localmente y se transmiten automáticamente cuando el enlace se restablece. Esto asegura que ninguna medición de espectro se pierda, independientemente de la calidad de la infraestructura de telecomunicaciones en el sitio de instalación.

10. Glosario de Términos

- **FFT:** Fast Fourier Transform.
- **IQ:** Muestras en cuadratura (In-phase / Quadrature).
- **PSD:** Power Spectral Density.
- **PFB:** Polyphase Filter Bank.
- **CS8:** Formato de datos complejo de 8 bits.
- **ZMQ (ZeroMQ):** Biblioteca de mensajería para comunicación entre procesos (IP-C/Red) de baja latencia.
- **Opus:** Códec de audio de baja latencia y tasa variable, usado comúnmente en streaming en tiempo real.
- **WebRTC:** Tecnología para transmisión en tiempo real (audio/video/datos) entre clientes, diseñada para baja latencia.
- **DTLS/SRTP:** Protocolos usados en WebRTC para cifrado y transporte seguro de audio/video.
- **Ring-Buffer:** Estructura circular de memoria para streaming continuo sin bloqueos prolongados.
- **SNR:** Signal-to-Noise Ratio (relación señal a ruido).